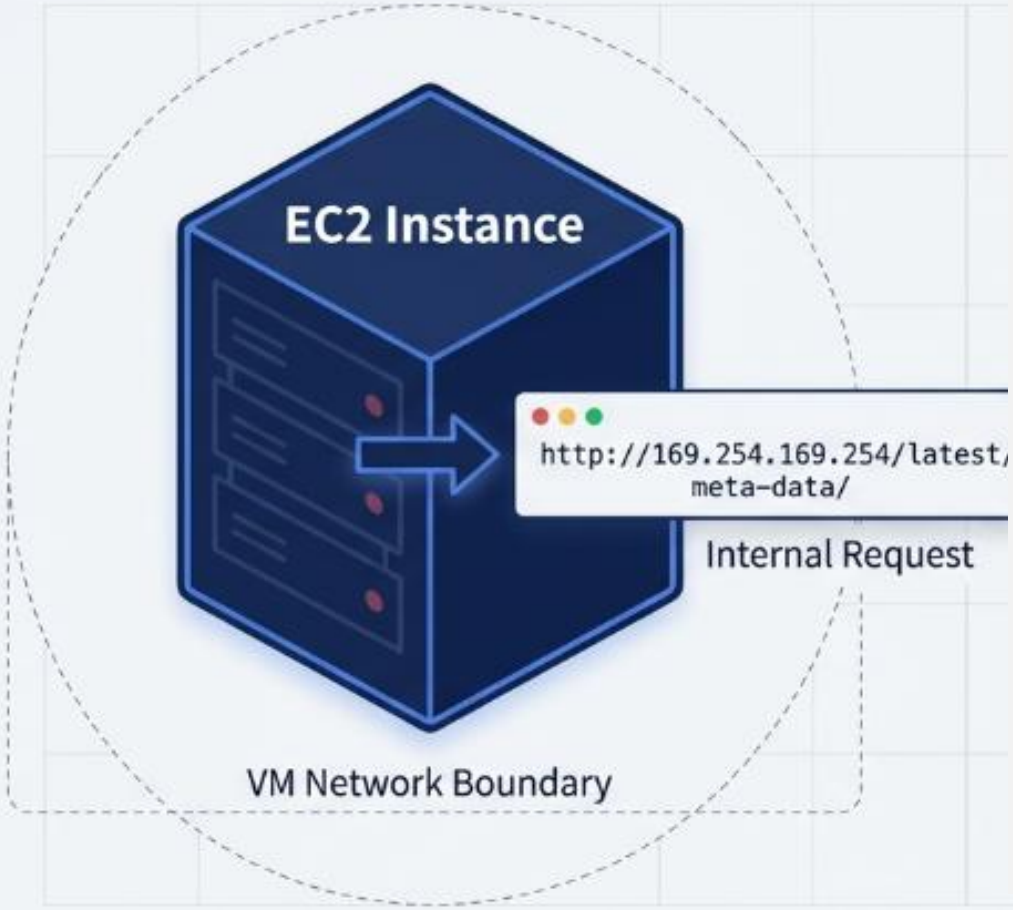
A02:2025 Security Misconfiguration

A02:2025 Security Misconfiguration



攻擊起點：
169.254.169.254

這是什麼？
這是雲端 VM（如 AWS EC2）內部的 Metadata Service。

它存了什麼？
🔑 IAM Role 憑證（Credentials）
🔓 Access Key
🔓 Secret Key
🔓 Session Token

A02:2025 Security Misconfiguration

```python
import requests
from flask import request

@app.route("/fetch")
def fetch():
    # ⚠️ 危險：直接接收使用者輸入的 URL
    url = request.args.get("url")
    return requests.get(url).text
```

Server-Side Request Forgery (SSRF)

致命的幾行程式碼：SSRF 漏洞

當 API 允許使用者傳入 URL 並由伺服器端發起請求 (SSRF) 時，災難就開始了。

A02:2025 Security Misconfiguration

攻擊鏈：從程式碼漏洞到帳號接管

攻擊者發送 Request

```
GET /fetch?url=http://169.254.169.254/latest/meta-data/iam/security-credentials/
```

伺服器回傳機敏憑證

```
{
  "Code": "Success",
  "Type": "AWS-HMAC",
  "AccessKeyId": "ASIAIOSFODNN7EXAMPLE",
  "SecretAccessKey": "wJalrXUtnFEMI/K7MDENG/bPxRfiCYEXAMPLEKEY",
  "Token": "IQoJb3JpZ2luX2Vj..."
}
```

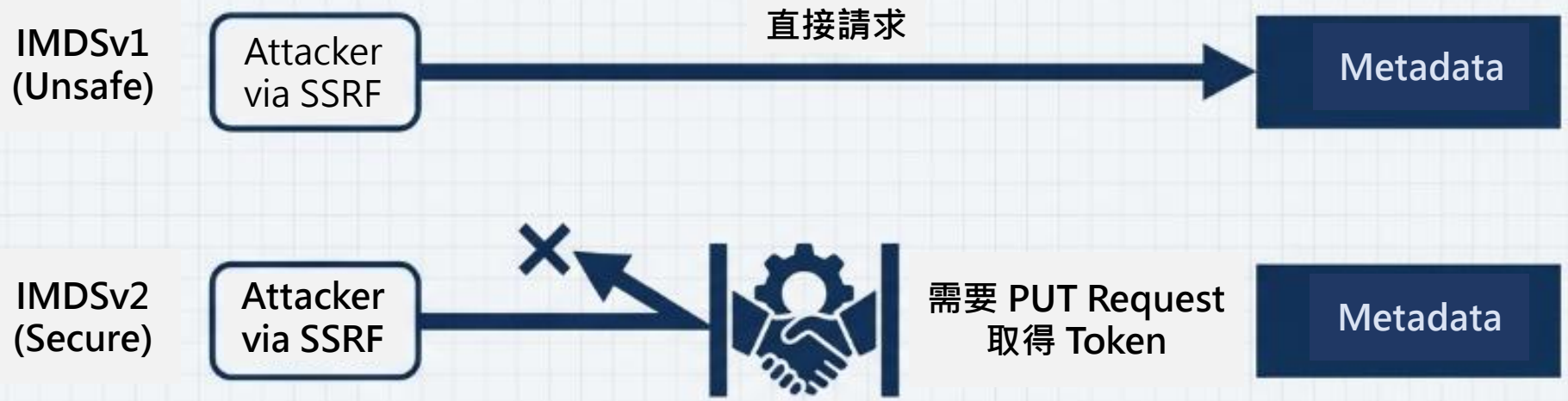
後果

攻擊者取得憑證後，可執行 aws s3 ls 或 aws ec2 describe-instances 控制整個帳號。

真實案例：2019 Capital One 資料外洩案（1 億筆個資外流）— 主因即為 WAF SSRF 加上 Metadata Service 未限制。

A02:2025 Security Misconfiguration

關鍵防禦：強制開啟 IMDSv2




```
aws ec2 modify-instance-metadata-options \
    --instance-id i-123456 \
    --http-tokens required
```

為什麼有效？IMDSv2 要求必須先透過 PUT request 取得 Session Token，一般的 SSRF 攻擊難以偽造此步驟。

強制 Token 驗證，SSRF 幾乎無法利用。

A02:2025 Security Misconfiguration

縱深防禦：最小權限原則（Least Privilege）

即使憑證外洩，也要將損害控制在最小範圍。

錯誤做法

```
{
  "Action": "*",
  "Resource": "*",
  "Effect": "Allow"
}
```

給予過大權限，導致整個帳號淪陷。

正確做法

```
{
  "Effect": "Allow",
  "Action": ["s3:GetObject"],
  "Resource": "arn:aws:s3:::my-bucket/*"
}
```

僅開放應用程式運作所需的特定權限。

A02:2025 Security Misconfiguration

給開發團隊的 3 個重點總結

檢視 SSRF 風險
只要 API 存在 SSRF 漏洞，攻擊者就能探測內部網路（169.254.169.254）。

強制啟用 IMDSv2
這是最有效的防禦，阻斷未經授權的 Metadata 存取。

實施最小權限（IAM）
不要使用 Action: *，確保應用程式權限最小化。

這不是理論，而是真實發生過的重大資安威脅。保護你的 Metadata，就是保護整個雲端帳號。

A02:2025 Security Misconfiguration

A02:2025 Security Misconfiguration: XXE 漏洞解析

深入理解 XML 外部實體攻擊與解析器陷阱

XML 不僅僅是資料格式，它是一種能執行指令的語言。
當 XML 解析器配置不當（Misconfiguration）時，
攻擊者可利用「外部實體（External Entities）」
建立通往伺服器內部的通道。

後果（Impact）
導致未經授權的檔案讀取（File Disclosure）、
伺服器端請求偽造（SSRF）或服務阻斷。

A02:2025 Security Misconfiguration

攻擊變體 I：標籤內容篡改（Tag Content Overwriting）

這是最直觀的 XXE 攻擊形式。
若應用程式會在回應（Response）中顯示 XML 的元素內容，
攻擊者即可利用外部實體讀取伺服器本機檔案。

惡意 Payload 定義 (Request)

Code Editor

```
<!DOCTYPE data [
    <!ENTITY s SYSTEM "etc/passwd">
]>
<data>
    <comment>
        <id>1123&s;</id>
    </comment>
</data>
```

引用外部實體

伺服器回應 (Response)

Terminal/Output

```
<response>
  Comment id=12345678
 root:x:0:0:root:/root:/bin/bash...
</response>
```

解析器處理 &; 時，將其替換為 /etc/passwd，內容並直接回傳。

A02:2025 Security Misconfiguration

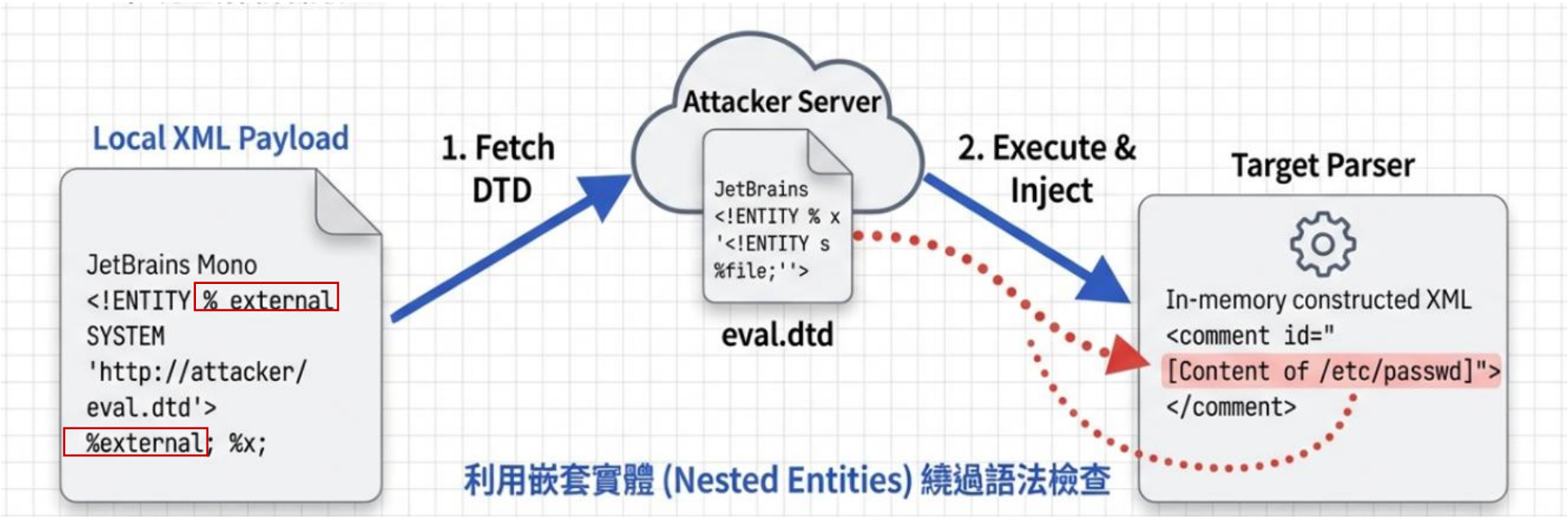
攻擊變體 II：屬性注入與參數實體的陷阱

- **技術限制：**

XML 規範通常禁止在「屬性值（Attribute Values）」中直接引用外部實體（例如：<comment id="&s;">）

- **繞過手法：**

攻擊者利用「參數實體（Parameter Entities，%name;）」配合「外部 DTD」來規避解析器檢查。

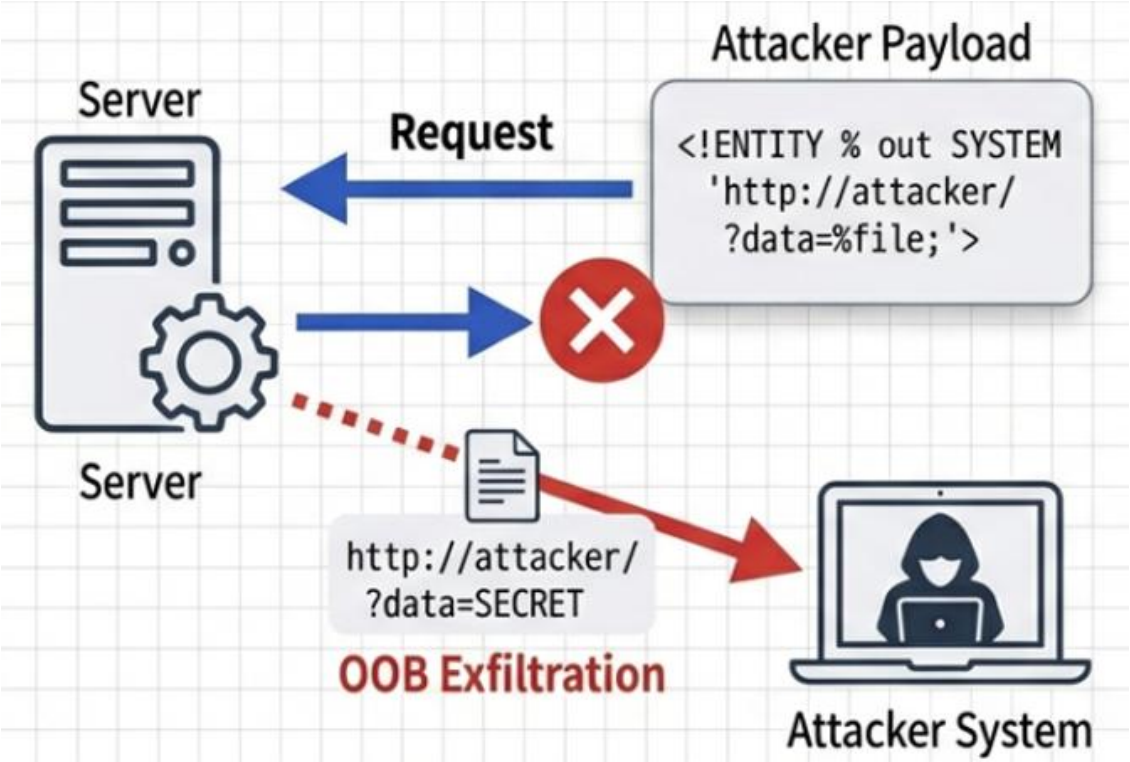


A02:2025 Security Misconfiguration

隱蔽攻擊（Blind XXE）與特定語言風險

Blind XXE 攻擊原理：

當伺服器不回傳數據時，攻擊者透過錯誤日誌（Error Logs）或誘發網路請求（Side Effects）來推斷內部資訊。



Java 環境風險

- Java 允許列出目錄內容 (Directory Listing)。
- Payload: file:///etc/
- 後果：可發現設定檔路徑，而非僅讀取特定檔案。

PHP 環境風險

- 支援 php:// 封裝協議 (Wrappers)。
- Payload: php://filter/convert.base64-encode/resource=檔案路徑
- 特點：可將二進位檔案轉為 Base64 讀取，繞過解析錯誤。

A02:2025 Security Misconfiguration

修復與防禦策略（Remediation）

核心哲學

- **防止 XXE 的關鍵在於正確配置 XML 解析器，而非過濾輸入。**

禁用外部實體加載 (Disable External Entity Loading)

最有效的防禦手段。配置解析器完全禁止 DOCTYPE 宣告。

建立白名單 (Whitelist Safe Entities)

若業務邏輯必須使用外部實體，僅允許特定、安全的實體來源。

解析器加固 (Security Hardening)

明確禁用 DTD（Document Type Definitions）。使用現代化、預設安全的 JSON/XML 庫。

Secure Configuration Example (Java)

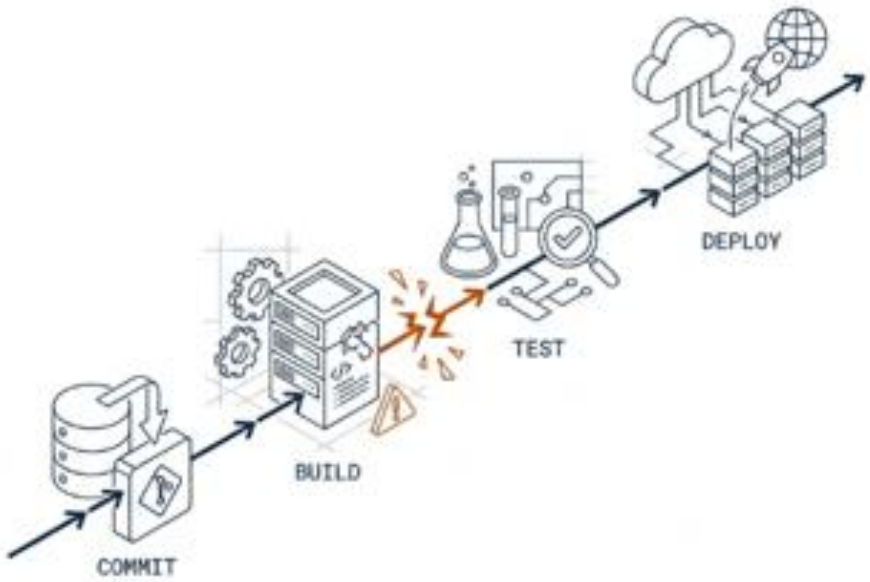
```java
DocumentBuilderFactory dbf =
DocumentBuilderFactory.newInstance();

// 禁用 DTD （最安全）
String FEATURE = "http://apache.org/xml/features/
disallow-doctype-decl";
dbf.setFeature(FEATURE, true);

// 若必須使用 DTD，則禁用外部實體
dbf.setFeature("http://xml.org/sax/features/external-
general-entities", false);
dbf.setFeature("http://xml.org/sax/features/external-
al-parameter-entities", false);
```

A03:2025 Software Supply Chain Failures

A03:2025 軟體供應鏈失效

Software Supply Chain Failures

從代碼提交到生產環境：防禦現代化開發流

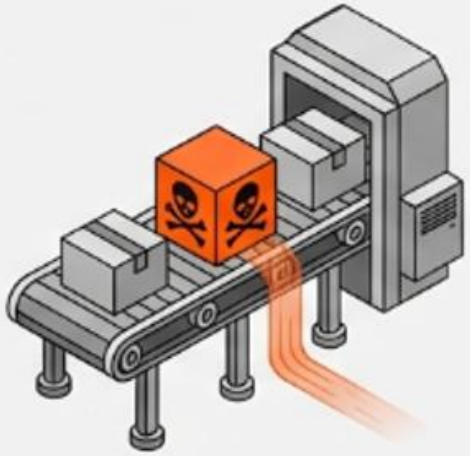


A03:2025 Software Supply Chain Failures

供應鏈攻擊的三大路徑

依賴套件劫持
針對開源庫，發布惡意「最新版本」。

未簽章的更新
攔截下載通道，替換為惡意檔案。



CI/CD 管道遭駭
從開發到部署的過程中下毒

A03:2025 Software Supply Chain Failures

風險演變與數據分析

2013: A9

Evolution

使用已知弱點的元件 (Known Vulnerabilities)。聚焦於單一有缺陷的函式庫 (focuses on single defective libraries).

2025: A03

軟體供應鏈失效 (Software Supply Chain Failures)。
範圍擴大至構建、分發與更新流程的崩潰 (scope expands to the collapse of build, distribution, and update processes).

50%

受訪者將此風險列為第一名 (#1) (Respondents rank this risk as #1)

5.19%

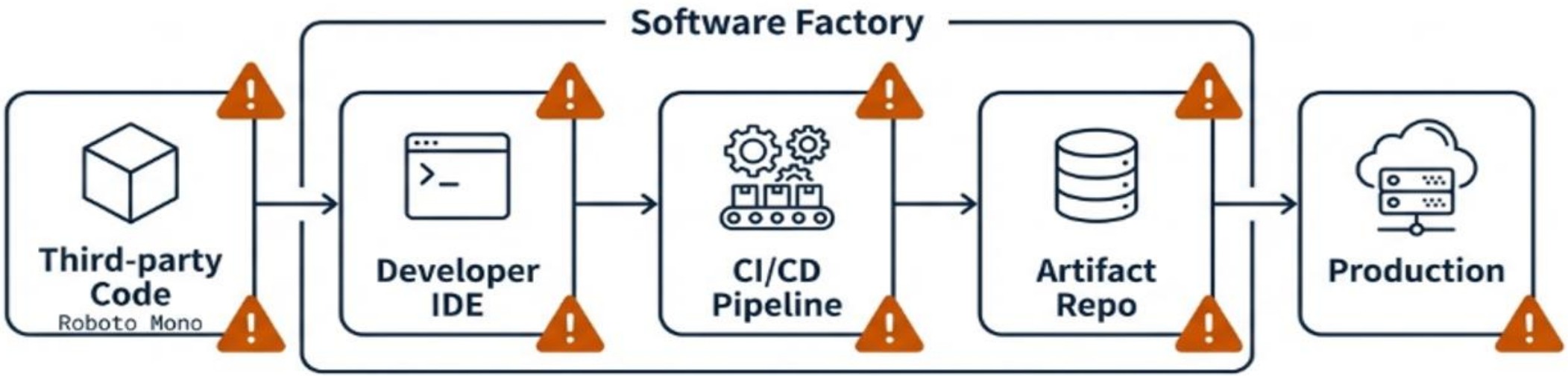
平均發生率 (Average Incidence Rate) - 資料集中最高 (Highest in dataset)

關鍵 CWE 指標

- CWE-477： Use of Obsolete Function
- CWE-1104：Unmaintained 3rd Party Components
- CWE-1329：Reliance on Component Not Updateable

A03:2025 Software Supply Chain Failures

攻擊面定義：不只是程式碼



定義
軟體供應鏈失效是指在軟體構建、分發或更新過程中的崩潰或妥協。

範圍
不僅限於應用程式代碼，包含 OS、Web 伺服器、DBMS、API、Runtime 環境及所有函式庫。

現實
攻擊者可能透過您系統依賴的第三方代碼、工具或相依性套件，植入惡意變更。

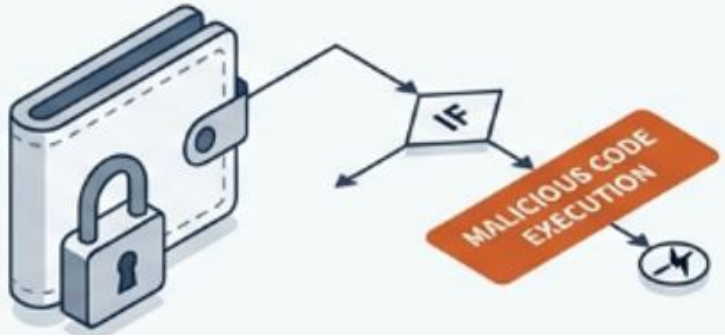
A03:2025 Software Supply Chain Failures

案例分析 I：受信任供應商的妥協

SolarWinds（2019） - 基礎設施攻擊

情境（Scenario）：
受信任的供應商遭植入惡意軟體。
機制（Mechanism）：
構建過程（Build Process）本身受到感染，
透過標準更新管道，導致約 **18,000 個組織** 受害。
影響（Impact）：
合法更新通道成為大規模入侵的載體。

Bybit Theft（2025） - 條件式惡意邏輯



情境（Scenario）：
錢包軟體供應鏈攻擊導致 **15 億美元** 盜竊。
機制（Mechanism）：
惡意代碼被設計為僅在特定條件下執行
（當目標錢包被使用時）。
教訓（Lesson）：
攻擊可以是潛伏且具備條件觸發的。

A03:2025 Software Supply Chain Failures

案例分析 II：針對開發者的蠕蟲與組件漏洞

Shai-Hulud（2025） - 自動傳播的 npm 蠕蟲

首個成功的自我傳播 npm 蠕蟲。
植入熱門套件的惡意版本。
使用安裝後腳本（post-install scripts）收集數據。
關鍵：偵測受害者環境中的 npm token，自動將惡意版本推送到其他套件。
規模：感染超過 **500 個套件版本**，證明開發者機器已成為主要目標。

組件漏洞

Log4Shell（CVE-2021-44228）
Apache Log4j 的遠端代碼執行（RCE）零日漏洞，
導致勒索軟體與挖礦攻擊。

Struts 2（CVE-2017-5638）
伺服器端任意代碼執行漏洞，
曾導致大規模資料洩露。

A03:2025 Software Supply Chain Failures

自我評估：您的團隊是否易受攻擊？

若符合以下任一情況，風險極高（High Risk）。

可視性不足（Visibility）

- 未追蹤所有元件及其傳遞性依賴（Transitive Dependencies）的版本。

缺乏修補（Patching）

- 軟體不再支援、過期，或未定期進行弱點掃描。

來源完整性（Integrity）

- 在生產環境中使用來源不明或未受信任的元件。

流程缺失（Process）

- 缺乏對 **IDE、擴充功能或 CI/CD 設定** 的變更管理機制。

權限過大（Access）

- 缺乏職責分離（Separation of Duty）。
- 單一人即可撰寫程式碼並直接部署至生產環境，且無人審核。

A03:2025 Software Supply Chain Failures

防禦支柱 I：依賴管理與可視化

1. 建立物料清單（SBOM）

集中生成並管理整個軟體堆疊的 SBOM。

必須追蹤巢狀 / 傳遞性依賴（Transitive Dependencies）。

2. 持續盤點與掃描

使用工具持續盤點客戶端與伺服器端元件。

推薦工具：

OWASP Dependency Track

OWASP Dependency Check

3. 情報監控

訂閱 CVE、NVD、OSV 警報。

利用 SCA 工具自動化此流程。

4. 清理（Clean Up）

移除未使用的依賴套件與功能，以此減少攻擊面。

Software Supply Chain Failures

防禦支柱 II：強化構建工廠

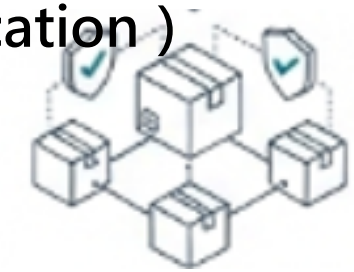
一、CI/CD 與構建伺服器

- 實施職責分離（Separation of Duties）
- 使用環境範圍的秘密（Environment-scoped secrets）
- 確保日誌不可篡改（Tamper-evident logs）
- 對所有產出物（Artifacts）進行數位簽章

二、開發者環境安全

- 強化 IDE 與工作站配置
- 定期修補與監控
- 全面啟用多因素驗證（MFA），並鎖定 IAM 權限

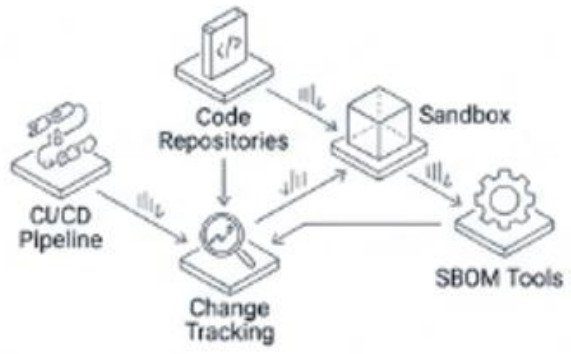
三、信任驗證（Trust Verification）

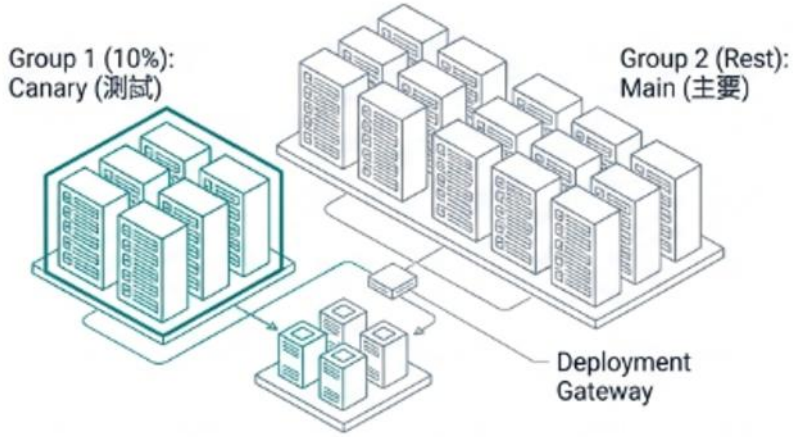


- 僅透過安全連結從官方 / 受信任來源獲取元件
- 優先使用已簽署的套件（Signed Packages），防止惡意篡改
- 採用 Zero Trust 模式：假設內部網路也可能受到威脅

A03:2025 Software Supply Chain Failures

防禦支柱 III：流程完整性與變更管理





一、變更管理（Change Management）

嚴格追蹤以下項目的變更：

- CI/CD 流水線設定
- 代碼庫（Code Repositories）
- 沙盒區域（Sandbox）
- SBOM 工具

二、部署策略與緩解（Deployment Strategy）

- **避免同步更新：**
  不要一次性將更新部署到所有系統。
- **金絲雀部署（Canary Deployments）：**
  採分階段發布，若受信任供應商遭入侵，可限制受害範圍（Blast Radius）。
- **虛擬補丁（Virtual Patching）：**
  若無法立即修補，部署虛擬補丁以監控或阻擋特定漏洞路徑。
- **相容性驗證：**
  開發者必須測試更新後函式庫的相容性，不可盲目自動更新。